BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

C PROGRAMMING

Unit 14

Function Part 4

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3
	1.1 Objectives	-	-	
2	Function with Arrays	-	1	4-10
3	Summary	-	-	11
4	Terminal Questions	-	-	12
5	Answers To Self Assessment Questions	-	-	12
6	Answers To Terminal Questions	-	-	12

1. INTRODUCTION

Functions with arrays in C programming involve passing arrays to functions as arguments, returning arrays from functions, and implementing algorithms that operate on arrays using functions. Passing arrays to functions allows for modular and reusable code by separating the array manipulation logic from the main program. This enables functions to perform specific tasks on arrays without modifying the original array directly. Returning arrays from functions allows functions to compute and generate arrays dynamically based on certain conditions or calculations. This provides flexibility in program design, as functions can generate arrays based on input parameters or other factors.

Additionally, implementing sorting algorithms like bubble sort, selection sort, and insertion sort using functions enhances code organization and readability. Functions encapsulate the sorting logic, making it easier to understand and maintain. By separating the sorting algorithms into functions, they can be reused across different parts of the program or in other programs.

Overall, functions with arrays enhance code modularity, reusability, and readability in C programming, making it easier to work with arrays and implement complex algorithms efficiently. They play a crucial role in developing robust and maintainable software solutions.

1.1. Objectives:

At studying this unit, you should be able to:

- *Learn how to pass arrays as arguments to functions.*
- *Implement algorithms like bubble sort, selection sort, Insertion sort, using functions.*

2. FUNCTION WITH ARRAYS

Functions with arrays in C programming involve passing arrays to functions as arguments and returning arrays from functions. This allows for modular and reusable code by separating the array manipulation logic from the main program.

Passing Arrays to Functions:

Arrays can be passed to functions as arguments.The function can then operate on the array elements without modifying the original array directly.

Syntax: void functionName(dataType arrayName[], int arraySize)

Returning Arrays from Functions:

Arrays can also be returned from functions. The function dynamically allocates memory for the array and returns a pointer to the array.

Syntax: dataType* functionName(dataType arrayName[], int arraySize)

Implementing Algorithms with Functions:

Functions with arrays can implement algorithms such as sorting (e.g., bubble sort, selection sort, insertion sort), searching, or other manipulations.

These algorithms operate on array elements and can be encapsulated in separate functions for better code organization.

Syntax: void bubbleSort(int array[], int arraySize)

Overall, functions with arrays enhance code modularity, reusability, and readability in C programming, making it easier to work with arrays and implement complex algorithms efficiently. They play a crucial role in developing robust and maintainable software solutions.

A C program that demonstrates passing an array to a function, modifying the array within the function, and then returning the modified array:

```
#include <stdio.h>

// Function to double each element of the array

void doubleArray(int array[], int arraySize) {

   for (int i = 0; i < arraySize; i++) {

      array[i] *= 2;

   }

}


int main() {

   int arr[] = {1, 2, 3, 4, 5};

   int arrSize = sizeof(arr) / sizeof(arr[0]);


   // Print the original array

   printf("Original array: ");

   for (int i = 0; i < arrSize; i++) {

      printf("%d ", arr[i]);

   }

   printf("\n");

   // Pass the array to the function to double its elements

   doubleArray(arr, arrSize);

   // Print the modified array

   printf("Modified array: ");

   for (int i = 0; i < arrSize; i++) {

      printf("%d ", arr[i]);
```

```
    }
    printf("\n");
    return 0;
}
```

This program defines a function doubleArray that takes an array and its size as arguments and doubles each element of the array. In the main function, an array arr is declared and initialized. The original array is printed, then passed to the doubleArray function to modify its elements. Finally, the modified array is printed to the console.

Implement algorithms like bubble sort, selection sort, Insertion sort, using functions.

Sorting in C refers to arranging elements in a specified order, usually either ascending or descending. There are various sorting algorithms available in C to accomplish this task, each with its own advantages and disadvantages. Here are some common sorting algorithms implemented in C:

Bubble Sort:

Bubble sort is a simple sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order.

Selection Sort:

Selection sort divides the input list into two parts: a sorted sublist and an unsorted sublist. It repeatedly selects the minimum element from the unsorted part and swaps it with the first element of the unsorted part.

Insertion Sort:

Insertion sort builds the final sorted list one item at a time by taking each element from the unsorted part and inserting it into its correct position in the sorted part.

This C program demonstrates the implementation of three different sorting algorithms: Bubble Sort, Selection Sort, and Insertion Sort.

```
#include <stdio.h>
// Bubble Sort
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
```

```
            if (arr[j] > arr[j + 1]) {
                // Swap arr[j] and arr[j+1]
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

// Selection Sort
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        // Swap arr[i] and arr[minIndex]
        int temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }
}

// Insertion Sort
void insertionSort(int arr[], int n) {
    int i, key, j;
    for (i = 1; i < n; i++) {
        key = arr[i];
```

```
        j = i - 1;

        // Move elements of arr[0..i-1], that are greater than key, to one position
ahead of their current position
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}

// Function to print an array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr) / sizeof(arr[0]);
    printf("Original array: ");
    printArray(arr, n);

    // Bubble Sort
    bubbleSort(arr, n);
    printf("Array sorted using Bubble Sort: ");
    printArray(arr, n);
```

```
    // Selection Sort
    int arr2[] = {64, 25, 12, 22, 11};
    selectionSort(arr2, n);
    printf("Array sorted using Selection Sort: ");
    printArray(arr2, n);

    // Insertion Sort
    int arr3[] = {64, 25, 12, 22, 11};
    insertionSort(arr3, n);
    printf("Array sorted using Insertion Sort: ");
    printArray(arr3, n);
    return 0;
}
```

SELF-ASSESSMENT QUESTIONS - 1

1. Which syntax is used to pass an array to a function in C?
   (a) void functionName(arrayType arrayName[], int arraySize)
   (b) void functionName(arrayType *arrayName, int arraySize)
   (c) void functionName(arrayType arrayName[], arraySize)
   (d) void functionName(arraySize, arrayType arrayName[])
2. When passing an array to a function, which parameter is used to specify the size of the array?
   (a) arrayType
   (b) arrayName
   (c) arraySize
   (d) functionType

3. How is an array returned from a function in C?

(a) By using the return statement with the array elements separated by commas

(b) By returning a pointer to the array

(c) By declaring the array as a global variable

(d) By passing the array as a parameter to the function

4. Which sorting algorithm is efficient for smaller data sets or nearly sorted arrays due to its adaptive nature?

(a) Merge Sort

(b) Quick Sort

(c) Insertion Sort

(d) Selection Sort

INSPIRED BY LIFE

3. SUMMARY

Functions with arrays in C programming enable modular and reusable code by allowing arrays to be passed to functions as arguments and returned from functions. This separation of array manipulation logic from the main program enhances code organization and readability. Additionally, implementing sorting algorithms like bubble sort, selection sort, and insertion sort using functions further enhances code modularity and reusability. By encapsulating the sorting logic within functions, it becomes easier to understand and maintain, and the functions can be reused across different parts of the program or in other programs. Overall, functions with arrays enhance code modularity, reusability, and readability in C programming, making it easier to work with arrays and implement complex algorithms efficiently, thereby contributing to the development of robust and maintainable software solutions.

INSPIRED BY LIFE

4. TERMINAL QUESTIONS

1. Implement the insertion sort algorithm in C and explain how it works.
2. Explain the process of sorting an array using the bubble sort algorithm.

5. ANSWERS TO SELF ASSESSMENT QUESTIONS

1. (a) void functionName(arrayType arrayName[], int arraySize)
2. (c) arraySize
3. (b) By returning a pointer to the array
4. (c) Insertion Sort

6. ANSWERS TO TERMINAL QUESTIONS

1. (Refer Section 2 for more details)
2. (Refer Section 2 for more details)